



Getting Started with Python

A comprehensive crash course in Python and computer science - from foundations to object-oriented programming, guided by Ryan Mitchell, a Python developer with over a decade of experience.

Facilitator: Okwunna Olive

Your Instructor

Meet Ryan Mitchell

About Okwunna Olive-Okafor

Okwunna Olive-Okafor has been writing Python for over a decade, building complex systems professionally every day. He'll guide you behind the scenes - showing not just *what* to type, but *why* it works.

What You'll Learn

- Data types and control flow
- Classes and object-oriented programming
- Parallel computing
- Modules and packages
- Everything you need to start coding

This Is a Real Course

Sometimes "introductory" means oversimplified - but that's not the case here. This is a genuine crash course in Python and computer science. If you're serious about learning Python but don't know where to start, you've found the right place.



WHO THIS COURSE IS FOR

Why Python?

Intuitive Syntax

Python reads almost like plain English, making it far easier to pick up than most languages.

Wide Popularity

Used across data science, web development, automation, and AI - Python skills are in high demand.

Gateway Language

Skills transfer easily to other languages, making Python an excellent first step into programming.

Are You Ready for This Course?

This course is designed for non-programmers - but some basic computer familiarity helps. Ask yourself:

→ File System Basics

Do you know where downloaded files go, what file extensions mean, and how to unzip files?

→ Terminal Awareness

Have you used your terminal or command line, even just a little?

→ Internet Fundamentals

Do you have a basic idea of how browsers fetch data from remote servers?

A strong background in logic and mathematics - or at least a willingness to learn - will also be a great asset.



Prior Programming Experience? Even Better

If you've done programming or front-end development before, some of this may be review - but understanding how these concepts work in other languages is a **huge asset**. This course builds from foundations upward, with each segment building on the previous one.

How to Get the Most Out of This Course

Watch Sequentially

Each segment builds on the last. Start from the beginning and work your way through.

Do the Challenges

Each challenge presents a bite-sized math or computer science concept programmers use every day.

Go at Your Own Pace

Pause, rewatch, give yourself time. This series is ~4½ hours - becoming proficient will take longer than that.

Set Aside Practice Time

Active coding practice between sessions accelerates learning dramatically.

Your Course Materials



Python Reference Guide

A quick reference to every major data type, function, and operation covered in the course. Keep it open as you code.



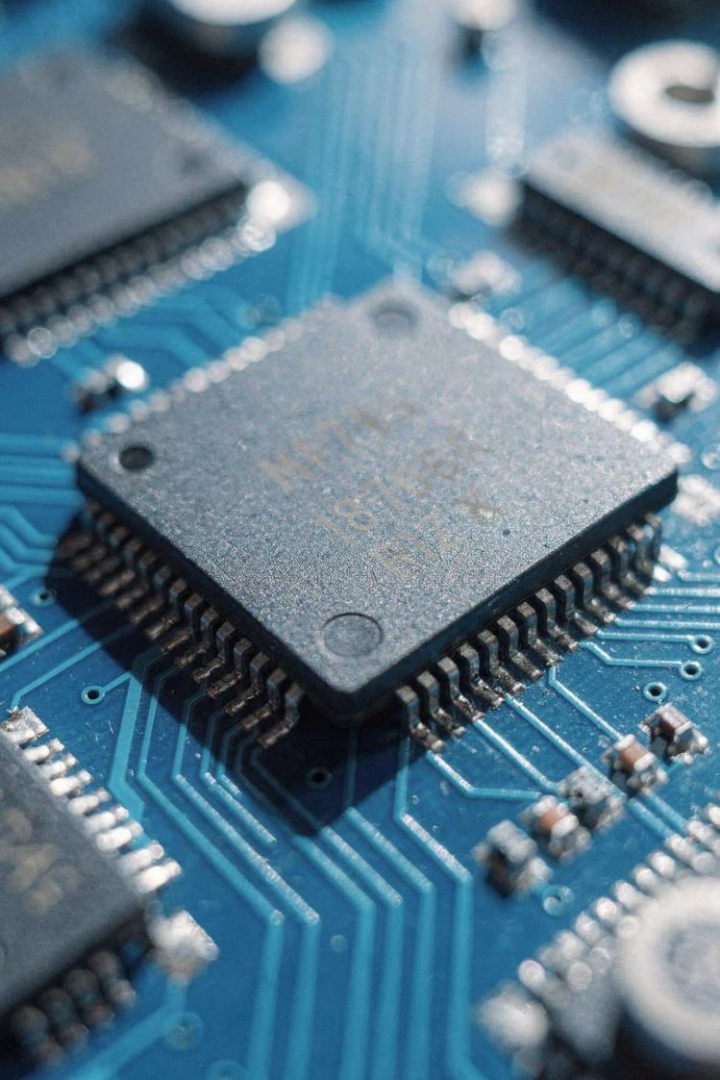
The Handout

A curated list of favourite Python learning resources, including advanced topics and complementary courses for after this one.



Exercise Files

Available on GitHub or in the Exercise folder. Each file corresponds to a module and section.



How Computers Think

When you save a file, it has a name and contents stored somewhere in memory. The file name points to a specific memory address - a unique number for every spot in memory. This pointer tells the computer exactly where to jump to retrieve the file's contents.

Pointers and Memory Addresses

What Is a Pointer?

A pointer is a memory address - literally a number - that tells the computer exactly where data is stored. The computer can jump directly to that location instantly.

How Files Are Read

Stored alongside the file name is the memory address of the file's contents and its size (e.g. 4 KB). The computer follows the pointer, reads the next 4 KB, and loads the content.

Variables Are Mini Files

When we write and run a program, we interact directly with the computer's memory. Programs create small bits of data called **variables** - they have names and point to data stored in memory.

```
a = 2  
b = 3  
c = a + b    # c = 5
```

The computer reads each line, stores values at memory locations, then fetches and combines them. Understanding this mental model is essential as programs grow in complexity.



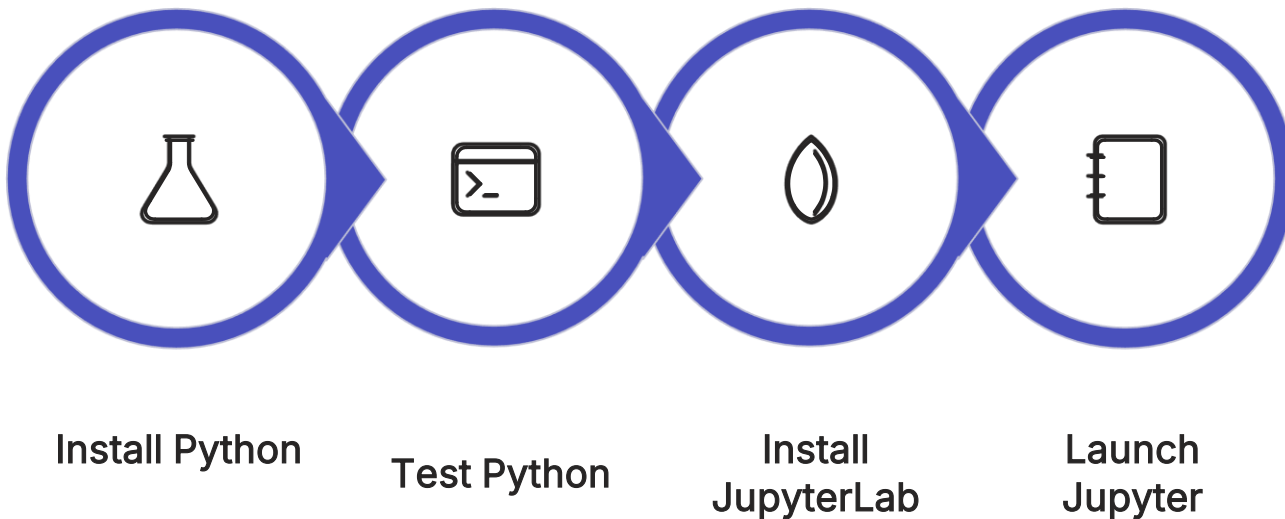
Important Concept: References vs. Copies

When you write `b = a` where `a` is a list, you are NOT duplicating `a`. Both `a` and `b` point to the *same* memory location. Modifying `a` will also change `b`.

```
a = [1, 2, 3, 4, 5]
b = a
a[0] = 6
# Both a and b are now [6, 2, 3, 4, 5]
```

This concept - that variables point to memory locations rather than holding copies of data - comes up constantly in Python programming.

Installing Python, Pip & JupyterLab



Follow each step carefully. A Troubleshooting document is included in the Exercise Files, and a Troubleshooting Wiki is available on GitHub for common issues.

Installing Python: Key Steps

01

Download Python

Go to python.org/downloads. The site auto-detects your OS and shows the latest version.

03

Test Your Installation

Open your terminal and type `python` (or `python3` on Mac). You should see the Python command prompt (`>>>`).

02

Windows: Customise Installation

Click **Customise Installation → Next**, then check **"Add Python to environment variables."** This step is critical.

04

Install JupyterLab

Exit the Python prompt, then type `pip install jupyterlab` and hit Enter.

Launching JupyterLab

Navigate to Your Files

Use the `cd` (change directory) command to navigate to where your Exercise Files are stored. Tip: copy the folder path from File Explorer or Finder and paste it into the terminal.

Start the Server

Type `jupyter lab` and hit Enter. A browser window will open showing your Exercise Files. Each `.ipynb` file is a Python Notebook you can run cell by cell.

 Visual Studio Code can also open and run `.ipynb` notebook files - a great alternative if JupyterLab installation causes issues. Download from code.visualstudio.com.



The Zen of Python

Python has been around for 30 years and its popularity keeps growing. It's an elegant, consistent language - the same syntax patterns repeat throughout, keeping complexity manageable even in large systems.

import this

Python's philosophy is captured in a hidden Easter egg. Open any terminal, enter the Python prompt, and type:

```
import this
```

"Beautiful is better than ugly. Explicit is better than implicit. Simple is better than complex. Complex is better than complicated."

This is your first mini challenge - run it yourself. Even if you don't understand every line now, revisit it at the end of the course and it will all make perfect sense.

FIRST PROGRAM

Writing Your First Python Program

Create hello.py

Open any text editor (VS Code, Sublime, PyCharm, Notepad++). Create a file called `hello.py` - `.py` is the extension for all Python files.

The Code

```
# print hello, world! to the terminal  
print('Hello, World!')
```

The `print` function outputs text to the terminal. Comments (starting with `#`) are ignored by the computer but useful for humans.

Running Your Program

1

Open Terminal

Open your terminal or command prompt.

2

Navigate to File

Use `cd` to navigate to where `hello.py` is saved. Use Tab to autocomplete directory names.

3

Run It

Type `python hello.py` and hit Enter. You should see `Hello, World!` printed to the screen.



Congratulations - you are now a Python programmer. Add it to your LinkedIn.



Jupyter Notebooks

Project Jupyter is a nonprofit that makes Python tools for programming and data science. Jupyter Notebook is a web application that lets you write and run Python in your browser - no command line required during use.

How Jupyter Works

Starting Jupyter

Running `jupyter lab` from the command line starts a local web server on your computer. A browser window opens automatically. If not, copy the URL printed in the terminal.

Notebooks & Cells

Each `.ipynb` file is a notebook made of cells. Each cell contains Python code you can run independently with **Shift + Enter**. Results appear directly below each cell.

Essential Jupyter Shortcuts

Shift + Enter

Run the current cell and move to the next. The most important shortcut - focus on this one first.

A / B

In command mode (click outside cell), press **A** to insert a cell above or **B** to insert below.

DD

In command mode, press **D** twice to delete the selected cell.

M

In command mode, press **M** to switch a cell to Markdown mode for formatted text and titles.

Variables: The Building Blocks

Variables are the most basic building block of any program - units assigned a value. The `=` sign is an **assignment operator**: it puts a value into a variable. Variable names must start with a lowercase letter and cannot begin with numbers or contain special characters (except underscores).



Python's Core Data Types

int

Whole numbers: 1, 42, -7. Use `type(x)` to confirm.

float

Decimal numbers: 1.5, 0.9. The decimal "floats" - it can appear anywhere in the number.

str

Strings of characters: "Ryan" or 'Hello'. A string of characters stored sequentially in memory.

bool

Boolean values: True or False (capital first letter). Usually produced by comparison operators.

complex

Complex numbers: 2j. Uses j notation (engineering convention). $1j * 1j = -1$.

Working with Strings

Concatenation

```
"String1" + "String2"
```

```
# → "String1String2"
```

```
"1" + "1"
```

```
# → "11" (not 2!)
```

Strings are not numbers - adding "1" + "1" gives "11", not 2.

Type Errors

```
"hello" + 1
```

```
# TypeError: can only concatenate
```

```
# str (not "int") to str
```

Always read error messages - Python tells you exactly what went wrong. Pay attention to variable types.

Booleans & Comparison Operators

```
1 == 1    # True   (comparison operator)
1 == 2    # False

x = 5     # assignment operator (single =)
x == 5    # True   (comparison, double ==)
```

 A single `=` assigns a value. A double `==` compares two values. Mixing these up is one of the most common beginner mistakes.

DATA STRUCTURES

Python Data Structures

Beyond single values, Python provides powerful data structures for storing collections of data. The four main ones are **lists**, **sets**, **tuples**, and **dictionaries** - each with distinct behaviours and use cases.



Lists

What Is a List?

A list is an ordered, mutable collection of values. Defined with square brackets. Can contain any mix of types - even other lists.

```
my_list = [1, 2, 3, 4]
my_list.append(5)
# → [1, 2, 3, 4, 5]
```

Key Properties

- Order matters - `[1,2] ≠ [2,1]`
- Can contain duplicates
- Mutable - you can add, remove, or change values
- Use `len(my_list)` to get the length

Sets & Tuples

Sets { }

Like lists, but all elements must be **unique** and **order does not matter**. Duplicates are silently discarded.

```
my_set = {1, 1, 2, 2}
# → {1, 2} (length: 2)
```

Tuples ()

Like lists, but **immutable** - once declared, they cannot be changed. Python allocates exactly the memory needed, making tuples more efficient for fixed data like X/Y coordinates.

```
my_tuple = (1, 2, 3)
# Cannot append or modify
```

Dictionaries

Dictionaries store **key-value pairs** - like a real dictionary where you look up a word and get its definition. Keys must be unique. Defined with curly brackets.

```
my_dict = {  
    "apple": "a red fruit",  
    "bear": "a scary animal"  
}  
my_dict["apple"] # → "a red fruit"
```

- ❗ If you add a second value for the same key, it overwrites the first. Both sets and dictionaries use curly brackets and require unique keys/values.

OPERATORS

Arithmetic Operators

+ Addition

`1 + 1 = 2`

* Multiplication

`3 * 4 = 12`

** Exponent

`5 ** 2 = 25`

/ Division

`20 / 5 = 4.0` (always returns a float)

% Modulus

Returns the remainder: `20 % 6 = 2`. Used constantly in programming.

String Operators

Concatenation (+)

```
"hello" + " world"  
# → "hello world"
```

Joins two strings together. Both operands must be strings
- mixing with integers causes a `TypeError`.

Repetition (*)

```
"ha" * 3  
# → "hahaha"
```

Repeats a string a given number of times. A handy trick for generating repeated patterns.

Comparison, Logical & Membership Operators

Comparison

`==`, `<`, `>`, `<=`, `>=` - compare two values and return a Boolean.

Logical

`and`, `or`, `not` - operate on Booleans. `not` flips a single Boolean value.

Membership

`in` and `not in` - check whether a value exists in a list or string. E.g.
`1 in [1,2,3] → True`.

 Note: `"cat" in "catatonic"` is `True` - be careful when using `in` with strings, as it matches substrings.



CONTROL FLOW

Control Flow: If Statements

Control flow is where programming becomes truly powerful. The `if` statement executes a block of code only when a condition is true. Indentation defines which lines belong to the block - this is critical in Python.

If / Else Blocks

```
a = True
if a:
    print("It is true")
    print("Also print this")
else:
    print("It is false")

print("Always print this") # outside the block
```

The colon followed by indentation defines a **block**. Lines at the same indentation level as the `if` statement always execute, regardless of the condition. Nested `if` statements are possible but can become unwieldy - loops help prevent excessive nesting.

For Loops

Iterating Over a List

```
my_list = [1, 2, 3, 4, 5]
for item in my_list:
    print(item)
```

The `for` loop iterates over any iterable (like a list). The variable name (`item`) is declared in the loop - you can name it anything.

Note on `in`

The `in` in a `for` loop is **not** the same as the membership operator `in`. In a `for` loop, it simply means "for each element in this iterable" - it's part of the loop syntax.

While Loops

```
a = 0
while a < 5:
    print(a)
    a = a + 1 # CRITICAL: increment the counter
```

⚠ Always ensure the condition in a `while` loop will eventually become `False`. Without the increment line, this loop would print `0` forever - an infinite loop. More than one programmer has made this mistake.

Use `while` loops when conditions are changing dynamically based on what happens inside the loop block.



FUNCTIONS

Functions: The Toaster Analogy

A function is like a toaster - you put something in (arguments), it applies a process, and returns a result. The same toaster can handle bread, pastries, or anything that fits. Functions let you define reusable processes in your code.

Defining and Calling Functions

```
def multiply_by_three(val):  
    return 3 * val
```

```
multiply_by_three(4)  # → 12
```

```
def multiply(val1, val2):  
    return val1 * val2
```

```
multiply(3, 4)  # → 12
```

Use the `def` keyword to define a function. Function names follow the same rules as variable names - start with a lowercase letter. The `return` keyword sends a value back to the caller.

Functions That Don't Return Values

Mutating Functions

```
def append_four(my_list):  
    my_list.append(4)  
  
a = [1, 2, 3]  
append_four(a)  
print(a) # → [1, 2, 3, 4]
```

None

Functions that don't explicitly return anything return `None` - Python's equivalent of null or undefined. `None` has its own type (`NoneType`). Be careful: operating on `None` (e.g. `None + 1`) will cause an error.

CLASSES & OOP

Classes and Object-Oriented Programming

What if you need dozens of related functions and variables - like a whole kitchen with a toaster, microwave, dishwasher, and coffee maker? A **class** is Python's tool for keeping related functions and attributes organised together.



Defining a Class

```
class Dog:
    def __init__(self, name):
        self.name = name
        self.legs = 4

    def speak(self):
        print(self.name + " says bark")
```

Class Names Use Uppercase

This distinguishes classes from variables and functions, which always start with lowercase.

`__init__` Is the Initialiser

Called automatically when a new instance is created.
`self` refers to the specific instance.

Creating and Using Objects

```
my_dog = Dog("Rover")  
another_dog = Dog("Fluffy")  
  
my_dog.speak()      # -> Rover says bark  
another_dog.speak() # -> Fluffy says bark
```

Objects

Instances of a class. `my_dog` and `another_dog` are both Dog objects.

Attributes

Variables inside a class, e.g. `self.name`, `self.legs`.

Methods

Functions inside a class, e.g. `speak()`. Called using dot notation.

Why Object-Oriented Programming?

Imagine building a whole house: a kitchen, laundry room, bathroom - multiple rooms, multiple appliances, each with their own settings and functions. Without classes, you'd end up with hundreds of functions and thousands of variables in an unmanageable file.

"A Python class is a great way to keep related collections of functions and attributes labelled and organised."

Classes let you model real-world complexity cleanly. This is just an introduction - their true power will become clear as the course progresses.

CHALLENGES

About the Challenges

In upcoming challenges, you'll work independently in Colab - just you and an open code editor. Each challenge presents a bite-sized math or computer science concept that programmers use every day.



If You Get Stuck - That's Normal

It's Not a Lack of Preparation

Writing a new program is a genuinely new way for your brain to work. Watching someone code and doing it yourself are completely different skills - like reading a novel vs. writing one.

Frustration Motivates Discovery

If you feel stuck, that's part of the process. Spend more time understanding the problem, run examples on paper, and use a search engine.

Use the Hints File

Every challenge includes a Python notebook with explicit nudges. Copy, run, and modify the code. Give yourself space and time.

Challenge Tips



Pause and Rewatch

This is not a university lecture hall. Short, fast-paced videos are there for you to use as needed - rewatch any section as many times as you like.



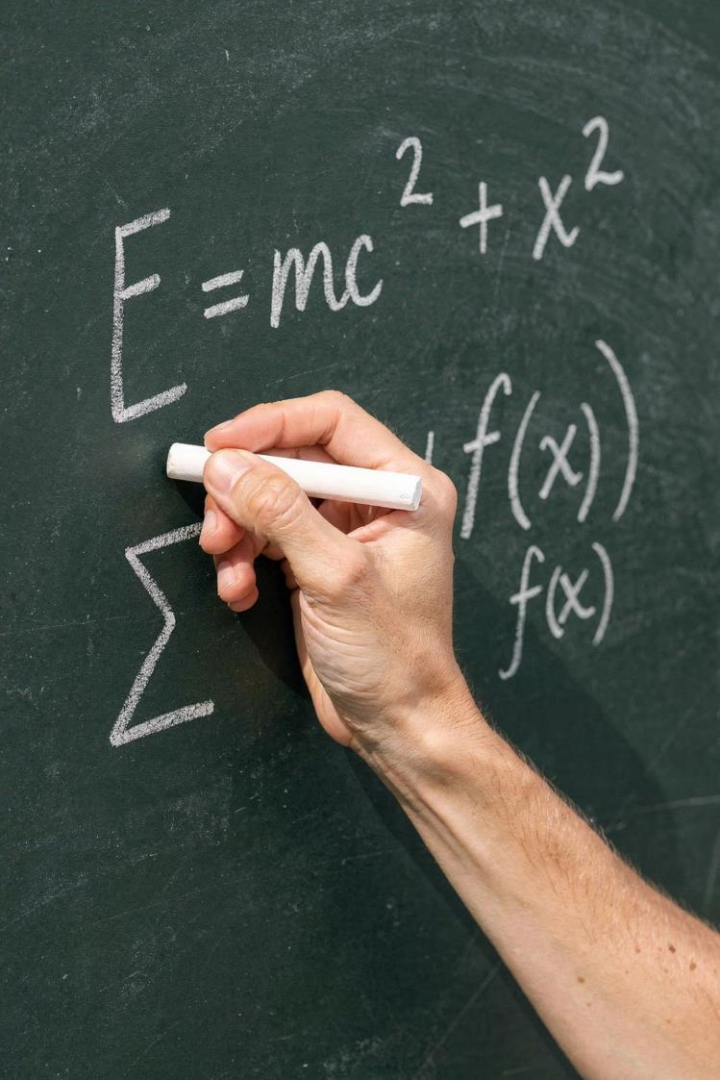
Write Anything

Write a function that just returns "Hello." Write anything. The goal is to get your brain used to this new skill.



Take Your Time

The series is ~4½ hours. If you're starting from scratch, becoming proficient will take longer than that - and that's completely fine.



CHALLENGE SOLUTION

Solution: Factorials

The factorial challenge tests input validation, loops, and edge cases. Here's the iterative solution - and a more elegant recursive one.

Factorial: Input Validation

```
def factorial(num):  
    if not isinstance(num, int):  
        return None  
    if num < 0:  
        return None
```

Non-integers Return None

Handles floats like 1.2 and strings like "spam" - both are valid test cases.

Negative Integers Return None

Negative integers are valid integers but have no factorial - a special case is required.

Factorial: Iterative Solution

```
fact = 1
counter = 1
while counter <= num:
    fact = fact * counter
    counter += 1
return fact
```

Starting with `fact = 1`, the loop multiplies by each integer from 1 up to `num`. When `num = 0`, the while loop never executes and `1` is returned - correctly, since mathematicians define $0! = 1$.

Factorial: Recursive Solution

```
def factorial(num):  
    if not isinstance(num, int): return None  
    if num < 0: return None  
    if num == 0: return 1  
    return num * factorial(num - 1)
```

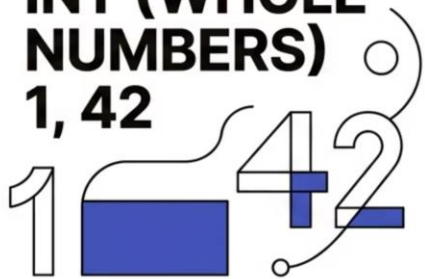
Recursion is a function calling itself. `factorial(3)` returns `3 × factorial(2)`, which returns `2 × factorial(1)`, which returns `1 × factorial(0)`, which returns `1`. So `factorial(3) = 3 × 2 × 1 × 1 = 6`.

- ❗ Recursion might seem like magic at first - but it's just a fancy kind of loop. It takes time to get the hang of, but it's a very handy skill to have.

Core Concepts: Data Types at a Glance

INT (WHOLE NUMBERS)

1, 42



FLOAT (DECIMALS)

1.5, 3.14



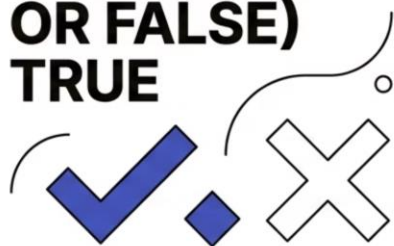
STR (TEXT STRINGS)

'HELLO'



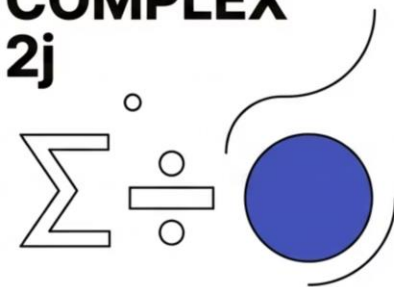
BOOL (TRUE OR FALSE)

TRUE



COMPLEX

2j



NONETYPE (ABSENCE)

NONE



Data Structures Compared

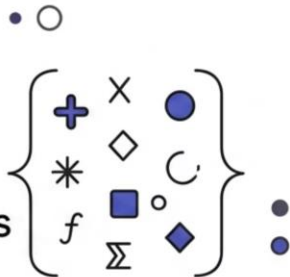
LIST

Ordered,
Mutable,
Allows duplicates



SET

Unordered,
Unique elements
only



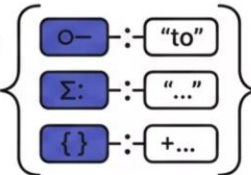
TUPLE

Ordered,
Immutable,
Memory-efficient



DICTIONARY

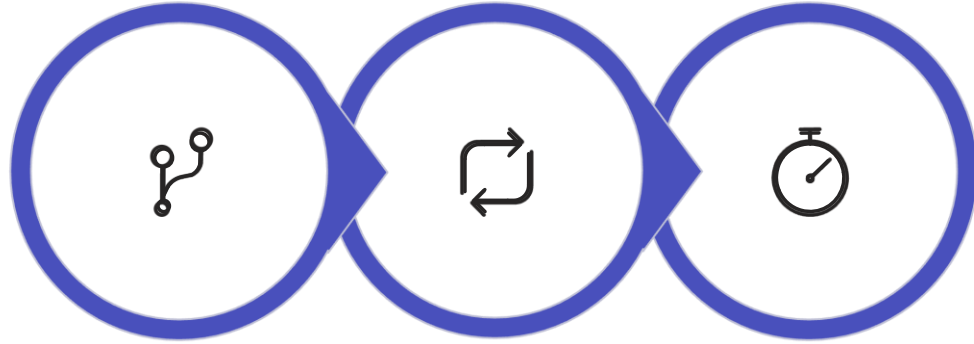
Key-Value pairs,
Unique keys



Operators Quick Reference

Category	Operators	Example
Arithmetic	<code>+ - * / ** %</code>	<code>20 % 6 = 2</code>
Comparison	<code>== != < > <= >=</code>	<code>5 >= 5 -> True</code>
Logical	<code>and or not</code>	<code>True and False -> False</code>
Membership	<code>in not in</code>	<code>1 in [1,2,3] -> True</code>
Assignment	<code>=</code>	<code>x = 5</code>

Control Flow Summary



If / Else

For Loop

While Loop

Control flow is what makes programs dynamic. Master these three structures and you can build the logic for almost any program.

Functions vs. Classes

Functions

- Defined with `def`
- Reusable blocks of logic
- Take arguments, return values
- Names start with lowercase
- Best for single, focused tasks

Classes

- Defined with `class`
- Bundle functions + attributes together
- Create objects (instances)
- Names start with Uppercase
- Best for modelling complex systems

The Python Learning Journey



Foundations

Variables, types, data structures, operators



Logic & Flow

Control flow, if/else, for and while loops



Functions & Classes

Reusable code, object-oriented programming



Advanced Python

Modules, packages, parallel computing, and beyond

Key Takeaways



Python Is Elegant

Consistent syntax, readable code, and a philosophy of simplicity make it the ideal first language.



Understand the Machine

Knowing how computers store and fetch data - pointers, memory, references - makes you a better programmer.



Practice Is Everything

Watching code being written and writing it yourself are completely different skills. Do the challenges.



Build Incrementally

Each concept builds on the last. Sequential learning, patience, and repetition are the path to proficiency.

Your Resources Checklist

- ☒ Download Python from python.org/downloads
- ☒ Install JupyterLab via `pip install jupyterlab`
- ☒ Download Exercise Files from GitHub
- ☐ Keep the Python Reference Guide open while coding
- ☐ Read the Handout for post-course learning resources
- ☐ Complete each challenge before moving to the next section
- ☐ Run `import this` in the Python prompt
- ☐ Revisit `import this` again at the end of the course



You've Got This

Python doesn't bite. Take it one step at a time, do the challenges, pause and rewatch as needed, and remember - every expert was once a beginner staring at a blank screen.

"If you're serious about learning Python, you'll get out of this course what you put into it."

- Ryan Mitchell

GOOD LUCK - AND WELCOME TO PYTHON PROGRAMMING.